

OK!

Obs: Só foi avaliado o que foi pedido.

PROJETO 3: PROJETO DE UM S.O. (BOS - BASIC OPERATING SYSTEM)

Gustavo Borges Peres da Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

Objetivos

- I/O: teclado;
- Tratamento de interrupção;
- Criação de um shell muito simples.

1. Tarefa 1: implementação de E/S para o teclado

1.1. Máquina utilizada

Para este projeto, será utilizado a mesma máquina hospedeira Windows 11 apresentada no tópico “1.1 Descrição” do projeto 1, cujo a mesma possui também um guest/convidado com a distro Debian Linux devido sua disponibilidade e licenciamento deste sistema que será utilizada para o andamento da tarefa 1, responsável por preparar o sistema e criar o primeiro kernel. Vale lembrar também, que nessa máquina possui dois usuários (so2 e vboxuser), portanto, o usuário que iremos utilizar é o vboxuser pois possui privilégios administrativos necessários para darmos continuidade.

1.2. Implementação de um Shell simples

Antes de iniciarmos, iremos abrir o nosso arquivo build.sh e adicionaremos um novo parâmetro à linha do gcc: -ffreestanding (ao final da linha). Essa flag vai evitar que o compilador use qualquer função oferecida pelo sistema hospedeiro, a não ser aquelas que definirmos durante a prática.

```
#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o -ffreestanding
gcc -m32 -c -w screen.c -o screen.o -ffreestanding
gcc -m32 -c -w string.c -o string.o -ffreestanding
gcc -m32 -c -w system.c -o system.o -ffreestanding
ld -m elf_i386 -T link.ld -o basic/boot/kernel.bin kasm.o kc.o screen.o string.o system.o
```

Código 1 – Conteúdo do arquivo script build.sh.

Agora iremos implementar uma sugestão realizada pelo professor, que se trata-se de

uma melhoria da função **print()** em **screen.h** que é a criação de uma variável **length** do tipo **uint8** para armazenar o valor de **strlen(ch)+1**. Com isso, evita-se que a cada execução do laço for se chame novamente a função **strlen()** para que essa obtenha o mesmo valor.

Por conta dessa sugestão, tivemos de adicionar as variáveis em **screen.h** e no código **screen.c** atualizamos a função **print()** conforme abaixo.

```
#ifndef SCREEN_H
#define SCREEN_H

#include "types.h"
#include "system.h"
#include "string.h"

extern int32 cursorX;
extern int32 cursorY;

void printch(char c);
void print(string ch);

#endif
```

Código 2 – Atualização do cabeçalho **screen.h**.

```
#include "basic/include/screen.h"

int32 cursorX = 0, cursorY = 0; // Posição do cursor na tela
const uint8 sw = 80, sh = 25, sd = 2; // Dimensões da tela (largura, altura, profundidade)

// Limpa a tela de uma linha específica até outra
void clearLine(uint8 from, uint8 to) {
    uint16 i = sw * from * sd;
    string vidmem = (string)0xb8000;
    for (i; i < (sw * (to + 1) * sd); i++) {
        vidmem[i] = 0x0; // Valor (caractere) 0
        vidmem[i+1] = 0x0F; // Cor branca sobre fundo preto
    }
}

// Atualiza a posição do cursor na tela
void updateCursor() {
    unsigned temp;
    temp = cursorY * sw + cursorX;
    outportb(0x3D4, 14);
    outportb(0x3D5, temp >> 8);
    outportb(0x3D4, 15);
    outportb(0x3D5, temp);
}

// Limpa toda a tela
void clearScreen() {
    clearLine(0, sh-1); // Chama clearLine p/ limpar da linha 0 até n (sh-1)
    cursorX = 0; // Reseta a posição do cursor na largura
    cursorY = 0; // Reseta a posição do cursor na altura
    updateCursor(); // Atualiza a posição do cursor
}
```

```

// Desloca conteúdo da tela para cima, remove 'lineNumber' linhas na parte superior
void scrollUp(uint8 lineNumber) {
    string vidmem = (string)0xb8000;
    uint16 i = 0;
    for (i; i < sw * (sh - 1) * sd; i++) {
        vidmem[i] = vidmem[i + sw * sd * lineNumber]; // Move o conteúdo da tela p/ cima
    }

    clearLine(sh - 1 - lineNumber, sh - 1); // Limpa linhas recém-criadas na parte inferior
    if ((cursorY - lineNumber) < 0) {
        cursorY = 0;
        cursorX = 0;
    } else {
        cursorY -= lineNumber; // Atualiza a posição do cursor na altura
    }
    updateCursor(); // Atualiza a posição do cursor
}

// Verifica se o cursor atingiu a última linha da tela, e se sim, rola a tela para cima
void newLineCheck() {
    if (cursorY >= sh-1) {
        scrollUp(1); // Chama scrollUp p/ rolar a tela p/ cima removendo uma linha
    }
}

// Imprime um caractere na posição atual do cursor
void printch(char c) {
    string vidmem = (string)0xb8000;
    switch(c) {
        case (0x08): // Retrocesso (BACKSPACE)
            if(cursorX > 0) {
                cursorX--; // Move cursor p/ posição anterior na largura
                vidmem[(cursorY * sw + cursorX)*sd] = 0x00; // Define o caractere como nulo
            } break;
        case (0x09): // Representa a tecla TAB
            cursorX = (cursorX + 8) & ~(8 - 1); // Move cursor p/ a próxima posição TAB
            break;
        case ('\r'): // Retorno de Carro
            cursorX = 0; // Move o cursor para a primeira posição na largura
            break;
        case ('\n'): // Nova Linha
            cursorX = 0; // Move o cursor para a primeira posição na largura
            cursorY++; // Move o cursor para a próxima linha na altura
            break;
        default:
            vidmem[((cursorY * sw + cursorX)) * sd] = c; // Define o caractere na tela
            vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F; // Define a cor do caractere
            cursorX++; // Move o cursor para a próxima posição na largura
            break;
    }
    if (cursorX >= sw) {
        cursorX = 0; // Se ultrapassar a largura, move o cursor para a próxima linha
        cursorY++; // na altura
    }

    newLineCheck(); // Verifica se o cursor atingiu a última linha da tela
    updateCursor(); // Atualiza a posição do cursor
}

// Imprime uma string chamando repetidamente printch para cada caractere na string
void print(string ch) {

```

```

uint8 length = strlen(ch) + 1; // Calcula o comprimento uma vez
uint16 i = 0;

for (i; i < length; i++) {
    printch(ch[i]); // Chama printch para cada caractere na string
}

```

Código 3 – Conteúdo do arquivo screen.c.

Com a sugestão implementada, agora daremos continuidade com a tarefa, cujo, para a implementação de um shell simples, teremos que criar a primeira função de entrada de dados, similar a função scanf() ou gets() em C. A função a ser criada se chama readStr() para a leitura de uma string. Além desta função, também será criada uma função acessório chamada strEql() a qual irá comparar duas strings. A função strEql() criada no cabeçalho (Código 4), será implementada em string.c (Código 5) cujo descrevemos o funcionamento da mesma através dos comentários.

```

#ifndef STRING_H
#define STRING_H
#include "types.h"

uint16 strlenh(string ch); // Função para calcular o comprimento de uma string
uint8 strEql(string ch1, string ch2); // Função para comparar duas strings

#endif

```

Código 4 – Conteúdo do cabeçalho string.h.

```

#ifndef STRING_H
#define STRING_H
#include "types.h"

uint16 strlenh(string ch); // Função para calcular o comprimento de uma string
uint8 strEql(string ch1, string ch2); // Função para comparar duas strings

#endif

```

Código 5 – Conteúdo do arquivo string.c

Para a criação da função readStr(), esta deve ser inserida dentro do arquivo kb.h, no diretório “include” (Código 6). E sua função foi implementada separadamente no código 7 para uma melhor organização.

```

vboxuser@Debian12:~/Projeto_2/so$ ls
basic      kasm.o      kernel.c    make.sh     screen.o    system.c
basic.iso  kc.o        kernel.o    run.sh      string.c    system.o
build.sh   kernel.asm  link.ld     screen.c    string.o
vboxuser@Debian12:~/Projeto_2/so$ nano basic/include/kb.h
vboxuser@Debian12:~/Projeto_2/so$ nano kb.c

```

Figura 1: Criando arquivo de cabeçalho kb.h e arquivo kb.c.

```

#ifndef KB_H
#define KB_H
#include "types.h"
#include "system.h"
#include "string.h"

// Função para traduzir os códigos de rastreamento de teclado para ASCII
string readStr();

#endif

```

Código 6 – Conteúdo do cabeçalho kb.h.

```

#include "basic/include/kb.h"

string readStr() {
    char buff[256];
    string buffstr = buff;
    uint8 i = 0;
    uint8 reading = 1;
    while(reading){
        if(inportb(0x64) & 0x1){
            switch(inportb(0x60)){
                /*case 1:
                    printch((char)27);
                    buffstr[i] =(char)27;
                    i++;
                    break;*/

                case 2:
                    ...

            }
        }
    }
    buffstr[i] = 0;
    return buffstr;
}

```

Código 7 – Conteúdo parcial do arquivo kb.c.

O objetivo desta função é traduzir os códigos de rastreamento de teclado (scancode) para código American Standard Code for Information Interchange (ASCII). Essa conversão é realizada no switch() que aparece no Código 7. Uma pequena amostra de como essa conversão deve ser realizada é apresentada dentro do switch(), para o scancode 1 (representando a tecla ESC) cujo a mesma está apresentada em comentário. Além disso, utilizaremos a tabela completa de scancodes disponibilizada pelo site Milliseconds [2] para termos a base de até quando teremos de implementar em nosso código que no caso foi proposto criar as entradas de conversão até a entrada 57 da tabela de scancodes.

Vale ressaltar que o **scancode** fornecido é para teclados americanos. E o teclado utilizado para este projeto é teclado mecânico HyperX HX 60 Alloy Origins AQU, Layout 60%, **US – americano** conforme a Figura 2. Os demais membros do grupo tiveram de testar outros layouts de teclados.



Figura 2: Teclado utilizado para o projeto.

Além disso, percebemos que no código de exemplo fornecido pelo professor (Codigo 7) a função de leitura é um “laço infinito”. Sendo assim, pensamos em utilizar uma condição de parada bem simples, cujo, ao pressionar ESC, atribuímos um valor igual a zero para a variável reading fazendo com que a leitura não seja prosseguida, logo, resultando-se na parada do laço.

*melhor
parar C/
Enter!*

Por outro lado, vale observar também que, para ler dados do teclado, é necessário ler dados de duas portas diferentes: 0x64 e 0x60.

- O endereço 0x64 permite ler o status do teclado, para saber quando uma tecla é pressionada.
- O endereço 0x60 permite a leitura do “valor” da tecla pressionada.

O `inportb(0x64) & 0x1` indica se uma tecla foi pressionada. Esta operação bitwise resultará em 1 para qualquer scancode válido (não existe scancode com valor 0). `inportb(0x60)` fornece o valor do scancode propriamente dito, o qual será convertido para código ASCII.

Conquanto, após toda essa análise, partimos para a finalização do nosso código 7 inserindo o restante das opções no `switch()` e condição de parada conforme podemos observar em nosso código completo/atualizado abaixo (Código 8).

```
#include "basic/include/kb.h"

string readStr() {
    char buff[256];
    string buffstr = buff;
    uint8 i = 0;
    uint8 reading = 1;
    while(reading){
        //0x64 = endereço da porta de status do controlador
        if(inportb(0x64) & 0x1){
            // 0x60 = endereço da porta de dados do controlador
            switch(inportb(0x60)){
                case 1: //ESC
```

```

        printch((char)27);
        buffstr[i] =(char)27;
        reading = 0;  // Encerra a leitura/loop
        break;
case 2: //1
    printch('1');
    buffstr[i] = '1';
    i++;
    break;
case 3: //2
    printch('2');
    buffstr[i] = '2';
    i++;
    break;
case 4: //3
    printch('3');
    buffstr[i] = '3';
    i++;
    break;
case 5: //4
    printch('4');
    buffstr[i] = '4';
    i++;
    break;
case 6: //5
    printch('5');
    buffstr[i] = '5';
    i++;
    break;
case 7: //6
    printch('6');
    buffstr[i] = '6';
    i++;
    break;
case 8: //7
    printch('7');
    buffstr[i] = '7';
    i++;
    break;
case 9: //8
    printch('8');
    buffstr[i] = '8';
    i++;
    break;
case 10: //9
    printch('9');
    buffstr[i] = '9';
    i++;
    break;
case 11: //0
    printch('0');
    buffstr[i] = '0';
    i++;
    break;
case 12: //-
    printch('-');
    buffstr[i] = '-';
    i++;
    break;
case 13: // =

```

```

        printch('=');
        buffstr[i] = '=';
        i++;
        break;
case 14: // BACKSPACE
        // Imprime char de controle '\b' para apagar na tela
        printch('\b');
        // Verifica se há chars na string antes de retroceder
        if (i > 0) {
            i--; // Retrocede indice na string
            buffstr[i] = 0; // Remove último char ao atribuir zero à posição
        }
        break;
case 15: //TAB
        printch('\t'); // Imprime o caractere de tabulação
        buffstr[i] = '\t'; // Armazena o caractere na string
        i++;
        break;
case 16: //q
        printch('q');
        buffstr[i] = 'q';
        i++;
        break;
case 17: //w
        printch('w');
        buffstr[i] = 'w';
        i++;
        break;
case 18: //e
        printch('e');
        buffstr[i] = 'e';
        i++;
        break;
case 19: //r
        printch('r');
        buffstr[i] = 'r';
        i++;
        break;

case 20: //t
        printch('t');
        buffstr[i] = 't';
        i++;
        break;
case 21: //y
        printch('y');
        buffstr[i] = 'y';
        i++;
        break;
case 22: //u
        printch('u');
        buffstr[i] = 'u';
        i++;
        break;
case 23: //i
        printch('i');
        buffstr[i] = 'i';
        i++;
        break;

```

atual na string


```
case 24: //o
    printch('o');
    buffstr[i] = 'o';
    i++;
    break;
case 25: //p
    printch('p');
    buffstr[i] = 'p';
    i++;
    break;
case 26: // [
    printch((char)91); // Código ASCII para '['
    buffstr[i] = '[';
    i++;
    break;
case 27: // ]
    printch((char)93); // Código ASCII para ']'
    buffstr[i] = ']';
    i++;
    break;
case 28: //ENTER
    printch('\n'); // Nova linha
    buffstr[i] = '\n';
    i++;
    break;
case 29: //CTRL
    //0x80 = usada para verificar se uma tecla foi liberada.
    if (inportb(0x60) & 0x80) {
        // Se a tecla 'Ctrl' foi liberada
        print("Ctrl Released");
    } else {
        // Se a tecla 'Ctrl' foi pressionada
        print("Ctrl Pressed");
    }
    break;
case 30: //a
    printch('a');
    buffstr[i] = 'a';
    i++;
    break;
case 31: //s
    printch('s');
    buffstr[i] = 's';
    i++;
    break;
case 32: //d
    printch('d');
    buffstr[i] = 'd';
    i++;
    break;
case 33: //f
    printch('f');
    buffstr[i] = 'f';
    i++;
    break;
case 34: //g
    printch('g');
    buffstr[i] = 'g';
    i++;
    break;
```

```

case 35: //h
    printch('h');
    buffstr[i] = 'h';
    i++;
    break;
case 36: //j
    printch('j');
    buffstr[i] = 'j';
    i++;
    break;
case 37: //k
    printch('k');
    buffstr[i] = 'k';
    i++;
    break;
case 38: //l
    printch('l');
    buffstr[i] = 'l';
    i++;
    break;
case 39: //;
    printch(';');
    buffstr[i] = ';';
    i++;
    break;
case 40: // '
    printch("\'");
    buffstr[i] = "\'";
    i++;
    break;
case 41: // `
    printch("`");
    buffstr[i] = "`";
    i++;
    break;
case 42: //LSHIFT
    if (inportb(0x60) & 0x80) {
        // Se a tecla 'Shift' foi liberada
        print("LShift Released");
    } else {
        // Se a tecla 'Shift' foi pressionada
        print("LShift Pressed");
    }
    break;
case 43: // \
    printch("\\");
    buffstr[i] = "\\";
    i++;
    break;
case 44: //z
    printch('z');
    buffstr[i] = 'z';
    i++;
    break;
case 45: //x
    printch('x');
    buffstr[i] = 'x';
    i++;
    break;
case 46: //c

```

```

        printch('c');
        buffstr[i] = 'c';
        i++;
        break;
case 47: //v
        printch('v');
        buffstr[i] = 'v';
        i++;
        break;
case 48: //b
        printch('b');
        buffstr[i] = 'b';
        i++;
        break;
case 49: //n
        printch('n');
        buffstr[i] = 'n';
        i++;
        break;
case 50: //m
        printch('m');
        buffstr[i] = 'm';
        i++;
        break;
case 51: // ,
        printch(',');
        buffstr[i] = ',';
        i++;
        break;
case 52: // .
        printch('.');
        buffstr[i] = '.';
        i++;
        break;
case 53: // /
        printch('/');
        buffstr[i] = '/';
        i++;
        break;
case 54: // RShift
        if (inportb(0x60) & 0x80) {
            // Se a tecla 'RShift' foi liberada
            print("RShift Released");
        } else {
            // Se a tecla 'RShift' foi pressionada
            print("RShift Pressed");
        }
        break;
case 55: // PRINTSCREEN - PrtSc
        // Tecla 'PrtSc'
        print("PrtSc");
        break;
case 56: // ALT
        if (inportb(0x60) & 0x80) {
            // Se a tecla 'Alt' foi liberada
            print("Alt Released");
        } else {
            // Se a tecla 'Alt' foi pressionada
            print("Alt Pressed");
        }
}

```

```

        break;
    case 57: // SPACE
        // Tecla 'Space'
        printch(' '); // Adiciona um espaço
        buffstr[i] = ' ';
        i++;
        break;
    }
}
buffstr[i] = 0;
return buffstr;
}

```

Código 8 – Conteúdo completo/atualizado do arquivo kb.c.

1.3. Testando o nosso shell simples

Agora para conseguirmos testá-lo, atualizaremos o arquivo kernel.c (código 9) e atualizaremos nosso script build.sh (código 10).

```

#include "basic/include/screen.h"
void kmain() {
    clearScreen();
    print(": : Digite algo:");

    // Chama a função readStr para obter a entrada do usuário
    updateCursor();
    string userInput = readStr();

    // Limpa a tela e imprime a string lida
    clearScreen();
    print(": : Voce digitou:");
    print(userInput);
    print("\n\napos ter pressionado ESC, o programa foi encerrado...");
}

```

Código 9 – Atualização do arquivo kernel.c.

```

#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o -ffreestanding
gcc -m32 -c -w screen.c -o screen.o -ffreestanding
gcc -m32 -c -w string.c -o string.o -ffreestanding
gcc -m32 -c -w system.c -o system.o -ffreestanding
gcc -m32 -c -w kb.c -o kb.o -ffreestanding
ld -m elf_i386 -T link.ld -o basic/boot/kernel.bin kasm.o kc.o screen.o string.o system.o kb.o

```

Código 10 – Atualização do script build.sh.

```

vboxuser@Debian12:~/Projeto_2/so$ tree
.
├── basic
│   ├── boot
│   │   ├── grub
│   │   │   └── grub.cfg
│   │   └── kernel.bin
│   └── include
│       ├── kb.h
│       ├── screen.h
│       ├── string.h
│       ├── system.h
│       └── types.h
├── basic.iso
├── build.sh
├── kasm.o
├── kb.c
├── kb.o
├── kc.o
├── kernel.asm
├── kernel.c
├── kernel.o
├── link.ld
├── make.sh
├── run.sh
├── screen.c
├── screen.o
├── string.c
├── string.o
├── system.c
└── system.o
5 directories, 25 files

```

Figura 3 – Visão geral da estrutura de arquivos. Criar pasta “so”.

Esse código do kernel.c vai fazer o seguinte, ele vai pedir ao usuário digitar algo, sendo assim, o usuário pode digitar qualquer coisa porém existem algumas limitações, tais como:

- Não foi implementado perfeitamente o uso de ALT, LSHIFT, RSHIFT, CTRL e PrtSc;
- Como não foi implementado o uso das teclas anteriormente ditas, não será possível usar combinações tais como ?, !, @, #, %, ^, & e etc.
- A tecla “\” funciona porém existe alguma barreira no QEMU que impede de mostrar esse símbolo no vídeo para nós. Tentamos usar a sintaxe `printch("\)`, porém o símbolo não é mostrado na input mas na output (ao pressionar ESC) funciona perfeitamente (o símbolo é visualizado).

Se pressionarmos ESC, o programa é finalizado, mas antes ele vai limpar toda a tela e retornar todo o conjunto de strings e parâmetros utilizados para o usuário verificar, vale ressaltar que, ao usar CTRL, Lshift, Rshift, Alt é mostrado uma mensagem de que a tecla foi pressionada, porém no output não será mostrado nada pois decidimos fazer absolutamente com esse valor no momento.

Então vamos lá compilar nosso código conforme a Figura 4.

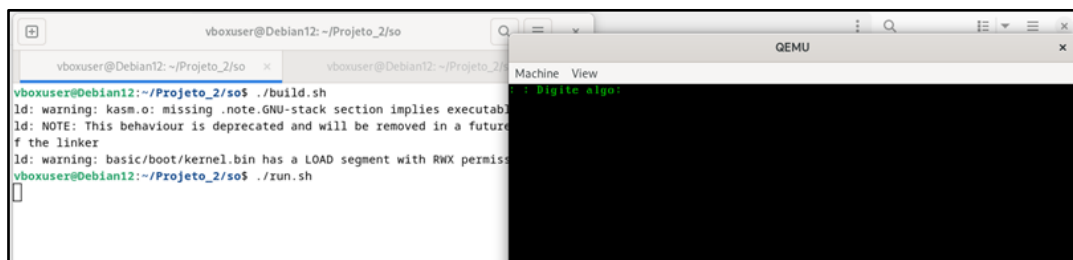


Figura 4 – Teste alfabeto.

Abaixo fizemos alguns testes para o entendimento do mesmo, sendo assim, começaremos com o alfabeto, utilizaremos a tecla ENTER para saltar uma linha, iniciaremos a linha com números de 0-9, saltaremos outra linha, e iniciaremos uma linha com símbolos em geral e por fim, saltaremos várias linhas porém cada linha eu utilizei uma tecla de atalho diferente para mostrar que a tecla foi pressionada de fato (Figura 5).

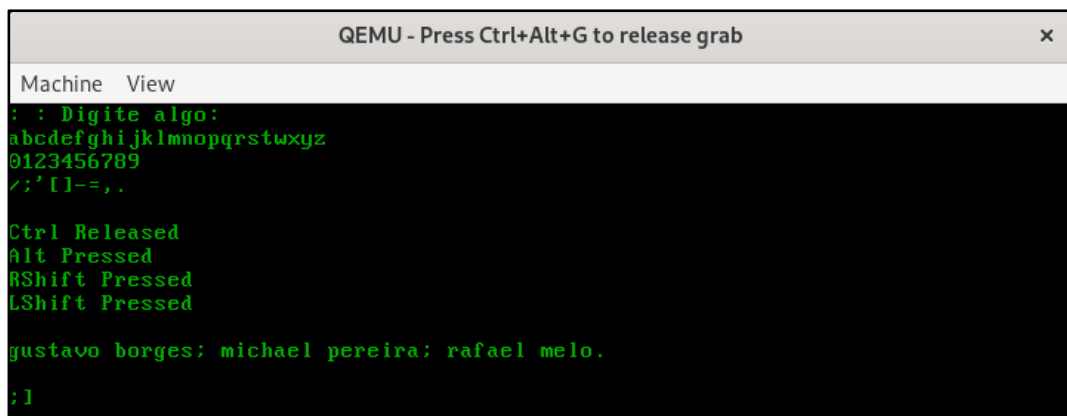


Figura 5 – Teste 1 do shell simples – INPUT de dados.

Agora ao pressionarmos ESC iremos verificar a nossa OUTPUT.

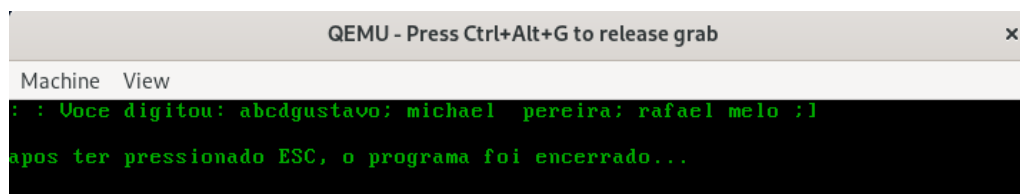


Figura 6 – Teste 1 do shell simples – OUTPUT de dados - FALHA.

Infelizmente não foi muito bom este teste que realizamos, então vamos fazer por partes. Sendo assim, dividiremos cada linha como um único teste conforme veremos abaixo.

Começaremos com um teste digitando apenas caracteres do alfabeto de a-z e demonstraremos a output para irmos direto ao ponto.

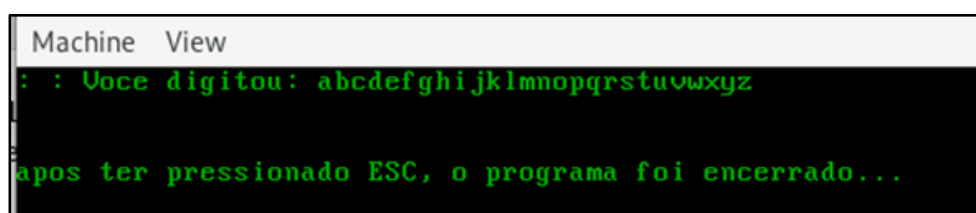
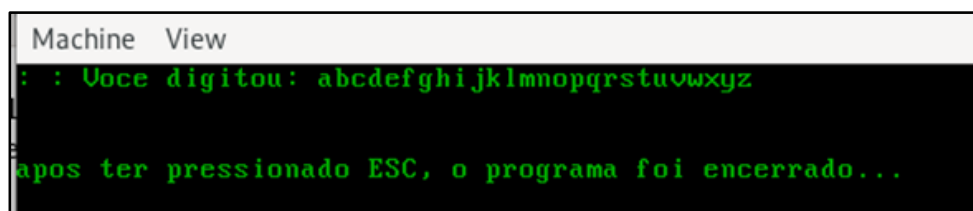


Figura 7 – Teste 2 – OUTPUT alfabeto.

Para cada teste, precisamos encerrar a nossa máquina QEMU pois nosso código encerra o programa e não existe outra interação a não ser fechar a janela e executar o script run.sh novamente.

Agora iremos fazer um teste digitando apenas números de 0-9.

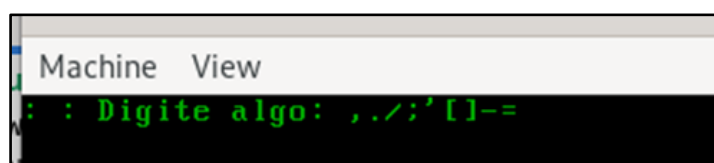


```
Machine View
: : Voce digitou: abcdefghijklmnopqrstuvwxyz
apos ter pressionado ESC, o programa foi encerrado...
```

Figura 8 – Teste 3 – OUTPUT números.

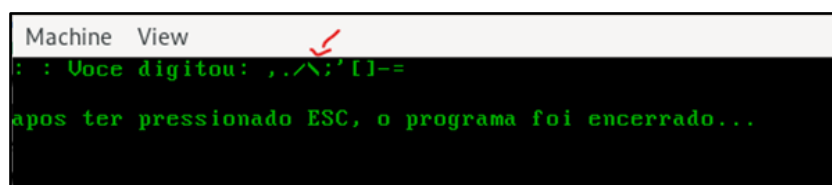
Agora iremos fazer um teste digitando apenas símbolos possíveis com a nossa configuração atual (temos combinações limitadas sem usar o CTRL, Rshift, Lshift e Alt).

Vale ressaltar que, nós havíamos dito que o uso de “\” por algum motivo não estava apresentando nos inputs, porém aqui já conseguimos observá-lo.



```
Machine View
: : Digite algo: ,./;\'[]-=
```

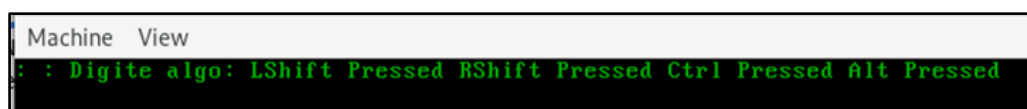
Figura 9 – Teste 4 – INPUT símbolos.



```
Machine View
: : Voce digitou: ,./;\'[]-=
apos ter pressionado ESC, o programa foi encerrado...
```

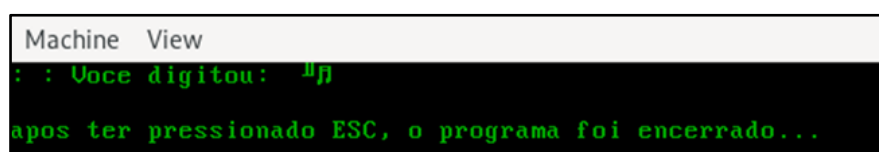
Figura 10 – Teste 4 – OUTPUT símbolos.

Agora testamos os atalhos CTRL, Rshfit, Lshift e Alt.



```
Machine View
: : Digite algo: LShift Pressed RShift Pressed Ctrl Pressed Alt Pressed
```

Figura 11 – Teste 5 – INPUT atalhos.



```
Machine View
: : Voce digitou:  apos ter pressionado ESC, o programa foi encerrado...
```

Figura 12 – Teste 5 – OUTPUT atalhos.

Tentaremos agora o uso de TAB e ENTER.

```
Machine View
: : Digite algo:      tab      tab      tab comando usado foi t
esse foi enter n
```

Figura 13 – Teste 6 – INPUT atalhos.

```
Machine View
: : Voce digitou:      tab      tab      tab comando usado foi \t
esse foi enter \n

apos ter pressionado ESC, o programa foi encerrado...
```

Figura 14 – Teste 6 – OUTPUT atalhos.

O teste foi finalizado, agora precisaremos fazer um arquivo ISO do nosso projeto. Porém para não ficar muito carregado a nossa tarefa, eu optei por pular as figuras, pois já foi explicado o método utilizado para fazermos o nosso arquivo ISO nas tarefas 1 e 2.

Em resumo, utilizaremos nosso script make.sh e iremos gerar uma ISO, esse arquivo iremos enviar para o nosso host através do protocolo SCP (Secure Copy Protocol). Com a ISO em mãos, mudaremos em nossas configurações da máquina virtual chamada BasicOS o seu disco óptico (“utilizaremos outro CD”), e após trocarmos para nossa “.iso”, estaremos com nosso sistema executando sem problemas conforme a Figura 15.

```
so x Gustafy x + v
> scp -r vboxuser@127.0.0.1:/home/vboxuser/Downloads/Projeto_3 "D:\\"
vboxuser@127.0.0.1's password:
string.o          100% 1352 284.1KB/s 00:00
make.sh           100% 43 14.0KB/s 00:00
system.c          100% 389 100.0KB/s 00:00
system.o          100% 1224 597.7KB/s 00:00
string.c          100% 720 351.6KB/s 00:00
basic.iso         100% 9294KB 46.3MB/s 00:00
kernel.asm        100% 144 70.3KB/s 00:00
run.sh            100% 60 29.3KB/s 00:00
kasm.o            100% 512 100.0KB/s 00:00
kb.o              100% 6348 3.0MB/s 00:00
link.ld           100% 131 42.6KB/s 00:00
build.sh          100% 383 187.0KB/s 00:00
kc.o              100% 1528 746.2KB/s 00:00
screen.o          100% 3492 1.7MB/s 00:00
system.h          100% 134 43.6KB/s 00:00
types.h           100% 293 143.1KB/s 00:00
screen.h          100% 196 65.5KB/s 00:00
kb.h              100% 188 36.7KB/s 00:00
string.h          100% 223 72.6KB/s 00:00
kernel.bin        100% 11KB 11.2MB/s 00:00
grub.cfg          100% 103 16.8KB/s 00:00
kernel.o          100% 1400 712.9KB/s 00:00
kernel.c          100% 421 137.0KB/s 00:00
screen.c          100% 3535 3.4MB/s 00:00
kb.c              100% 6244 3.0MB/s 00:00
setup.sh          100% 547 133.6KB/s 00:00

MEM: 32% | 26/796B | 3s 181ms
03:42 | #
```

Figura 15 – Copiando arquivos entre guest e hospedeiro.

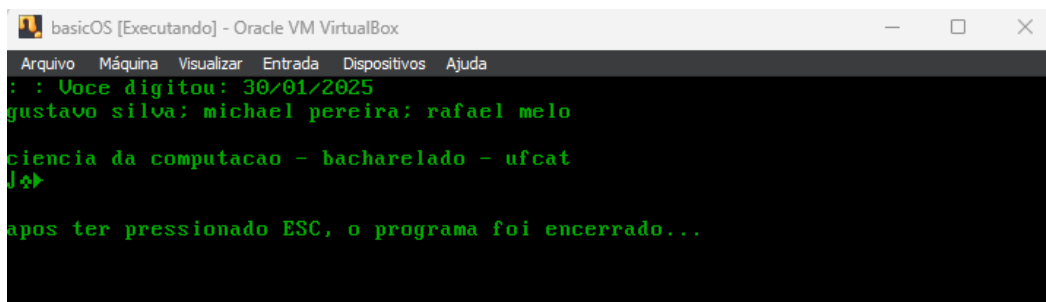


Figura 16 – Testando nossa ISO em nossa Máquina Virtual.

Um adendo importante, é que enquanto o relatório estava sendo desenvolvido, alteramos o nome da pasta raiz de “Projeto_2” para “Projeto_3”.

2. Tarefa 2: implementação de interrupções

(30/01/2025 a 02/02/2025).

3. Tarefa 3: modificação da estrutura do projeto e build

(03/02/2025 a 06/02/2025).

4. Tarefa 4: construção de um shell simples

(06/02/2025 a 09/02/2025).

5. Considerações finais

(06/02/2025 a 09/02/2025).

6. Referências

[1] Dalton Matsuo Tavares, (2025) “Projeto 3 – Projeto de um S.O. (BOS - Basic Operating System)”, material disponibilizado no google classroom e SIGAA, Dezembro.

[2] Milliseconds, (2023) “Keyboard Scan Codes”, <https://www.millisecond.com/support/docs/v5/html/language/scancodes.htm>, Dezembro.